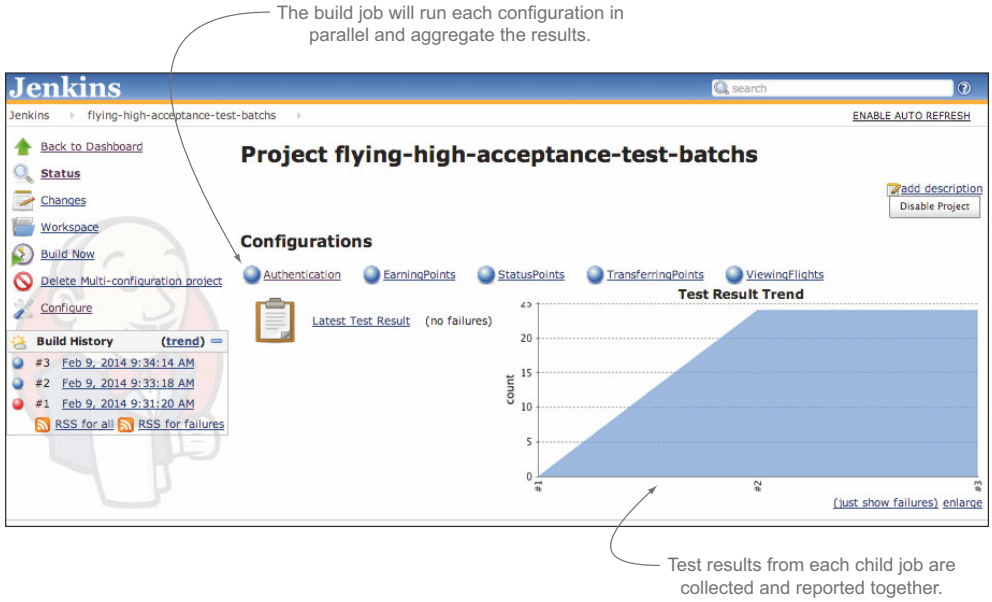


Figure 12.11 The results of each child job are displayed on the main job page.

Selenium Grid lets you set up remote machines that you can use to farm out automated web tests. Teams often use Selenium Grid to run web tests on different browsers, devices, and operating systems, or to distribute the load in large automated web testing suites. You can use Selenium Grid to run tests on browsers on Windows, Linux, and OS X, and even on mobile devices running Android and iOS.

Unlike the CI-based approach we looked at earlier, the tests are often executed on a single machine (typically the build server). But when the web tests are executed, they won't open a browser locally; instead, they'll delegate to the Selenium Grid server, which will open and control a browser on a remote machine (see figure 12.12). You can configure any browser supported by the operating system on the node machine, including PhantomJS.

Note that although Selenium Grid does make it easy to run web tests on different operating systems and browsers, it won't in itself run the web tests in parallel. For this to happen, you still need to configure the tests to run in parallel, such as by using the strategy discussed in section 12.5.1.

A Selenium Grid server is easy to set up.[2] The first thing you'll need is a machine to act as your Selenium Grid Hub. Choose one with a decent amount of memory and CPU power—although this machine doesn't run the browsers itself, WebDriver tests are memory-hungry, and if the server doesn't have enough resources, the tests will fail unexpectedly.

[2] See the Selenium Grid2 wiki for detailed instructions: https://code.google.com/p/selenium/wiki/Grid2.